

EventHub

Plataforma de Eventos e Venda de Ingressos

Documentação técnica e requisitos do projeto
Projeto de estudo para evolução back-end (Java / Spring Boot)

Stack: Java 21 · Spring Boot 3 · PostgreSQL · Redis · Kafka

Versão do documento: 1.0

Data: Maio de 2026

Sumário

1. Visão geral do projeto
2. Objetivos de aprendizado
3. Requisitos funcionais
4. Requisitos não funcionais
5. Atores e papéis (roles)
6. Modelo de domínio
7. Arquitetura e organização do código
8. Stack tecnológico
9. Endpoints principais (API)
10. Roadmap de implementação por fases
11. Critérios de aceitação
12. Entregáveis e documentação
13. Próximos passos sugeridos

1. Visão geral do projeto

O **EventHub** é uma plataforma back-end para criação e gestão de eventos com venda de ingressos online. Organizadores cadastram eventos, definem lotes de ingressos com preço e quantidade, e clientes realizam a compra. A confirmação de pagamento é simulada de forma assíncrona via mensageria, e cada ingresso pago é emitido com um código único que pode ser validado em um endpoint de check-in.

O projeto foi desenhado especificamente como projeto de estudo para evolução do nível júnior para pleno. Ele cobre, de forma incremental, os principais temas exigidos em vagas back-end intermediárias: APIs REST, autenticação, banco relacional avançado, cache, mensageria, concorrência, testes automatizados e containerização.

Por que esse projeto

Diferente de um CRUD genérico, o EventHub tem regras de negócio reais (estoque limitado de ingressos, prazos de pagamento, validação de check-in) e problemas clássicos de back-end (concorrência na venda do último ingresso, consistência entre banco e mensageria, idempotência). Isso o torna um projeto fácil de defender em entrevistas técnicas, pois cada decisão pode ser justificada.

2. Objetivos de aprendizado

Ao concluir o projeto, espera-se domínio prático dos seguintes tópicos:

- Construção de APIs REST robustas com Spring Boot (validação, tratamento de erros, paginação).
- Autenticação e autorização com Spring Security e JWT.
- Modelagem relacional, otimização de consultas SQL e uso correto de índices.
- Migrations de banco de dados com Flyway.
- Cache distribuído e rate limiting com Redis.
- Mensageria assíncrona com Kafka, incluindo idempotência, retry e DLQ.
- Controle de concorrência (locks pessimistas e locks distribuídos).
- Testes unitários e de integração com JUnit, Mockito e Testcontainers.
- Containerização com Docker e orquestração local via Docker Compose.
- Documentação de API com OpenAPI/Swagger.

3. Requisitos funcionais

3.1 Gestão de usuários

- RF-01. Permitir o cadastro de novos usuários com nome, e-mail e senha.
- RF-02. Autenticar usuários via login, retornando um access token (JWT) e um refresh token.
- RF-03. Permitir a renovação do access token a partir do refresh token.

- RF-04. Senhas devem ser armazenadas com hash BCrypt; nunca em texto puro.
- RF-05. Cada usuário possui um papel: ADMIN, ORGANIZER ou CUSTOMER.

3.2 Gestão de eventos

- RF-06. Usuários com papel ORGANIZER podem criar, editar e cancelar seus próprios eventos.
- RF-07. Um evento contém: título, descrição, local, data/hora, status e organizador.
- RF-08. O status do evento segue o ciclo: DRAFT → PUBLISHED → (CANCELLED | FINISHED).
- RF-09. Apenas eventos com status PUBLISHED são visíveis na listagem pública.
- RF-10. A listagem pública deve suportar paginação, ordenação e filtros por cidade, intervalo de datas e busca por texto no título.
- RF-11. ADMIN tem acesso de leitura a eventos em qualquer status.

3.3 Lotes de ingressos

- RF-12. Um evento pode ter um ou mais lotes (ex.: Pista, VIP, Camarote).
- RF-13. Cada lote possui: nome, preço, quantidade total, quantidade disponível, data de início e fim de vendas.
- RF-14. Apenas o organizador do evento pode criar e editar seus lotes.
- RF-15. Não é permitido reduzir a quantidade total abaixo da quantidade já vendida.

3.4 Compra de ingressos

- RF-16. CUSTOMER pode criar um pedido contendo um ou mais itens (lote + quantidade).
- RF-17. Ao criar o pedido, a quantidade solicitada deve ser reservada e a quantidade disponível do lote deve ser decrementada de forma atômica.
- RF-18. O pedido nasce no status PENDING com prazo de expiração configurável (padrão: 10 minutos).
- RF-19. Não é permitido comprar ingressos de evento que não esteja PUBLISHED.
- RF-20. Não é permitido comprar fora da janela de vendas do lote.
- RF-21. A confirmação do pagamento é processada de forma assíncrona, atualizando o pedido para PAID ou CANCELLED.
- RF-22. Pedidos PENDING não pagos no prazo devem ser expirados automaticamente e a quantidade reservada devolvida ao lote.

3.5 Emissão e check-in de ingressos

- RF-23. Ao confirmar o pagamento, um ingresso individual deve ser emitido para cada unidade adquirida, com código UUID único e status VALID.
- RF-24. O endpoint de check-in valida um código de ingresso e altera seu status para USED, registrando data e hora.
- RF-25. Não é permitido fazer check-in de ingresso já USED, CANCELLED ou inexistente.

- RF-26. O check-in deve ser seguro contra concorrência (mesmo código não pode ser usado duas vezes em chamadas simultâneas).

4. Requisitos não funcionais

- RNF-01. A aplicação deve ser stateless: nenhum estado de sessão em memória.
- RNF-02. Todas as senhas e segredos devem ser injetados via variáveis de ambiente.
- RNF-03. Logs estruturados, com nível configurável e correlação por requisição.
- RNF-04. A API deve responder em formato JSON e usar Problem Details (RFC 7807) para erros.
- RNF-05. Endpoints públicos de listagem devem ter cache no Redis e suportar invalidação.
- RNF-06. O endpoint de login deve possuir rate limiting (proteção contra brute force).
- RNF-07. Consumidores Kafka devem ser idempotentes e ter retry com Dead Letter Queue.
- RNF-08. Cobertura de testes deve focar em regras de negócio e fluxos críticos, não em métricas percentuais artificiais.
- RNF-09. A aplicação deve subir localmente apenas com 'docker compose up' + './mvnw spring-boot:run'.
- RNF-10. Documentação OpenAPI exposta em /swagger-ui.html.

5. Atores e papéis (roles)

Papel	Permissões
ADMIN	Acesso total. Lê qualquer evento, gerencia usuários, remove conteúdo impróprio.
ORGANIZER	Cria e gerencia seus próprios eventos e lotes. Consulta seus pedidos e relatórios.
CUSTOMER	Lê eventos publicados, faz compras, gerencia seus ingressos e realiza check-in.
Anônimo	Lê apenas a listagem pública de eventos.

6. Modelo de domínio

O domínio é composto por seis entidades principais. Os relacionamentos são propositalmente simples para manter o foco em conceitos back-end sem a complexidade de DDD ou arquitetura hexagonal.

6.1 Entidades

Entidade	Campos principais
User	id, name, email (único), passwordHash, role, createdAt
Event	id, title, description, location, dateTime, status, organizerId, createdAt, updatedAt, deletedAt
TicketBatch	id, eventId, name, price, totalQuantity, availableQuantity, saleStartAt, saleEndAt
Order	id, userId, status, totalAmount, createdAt, expiresAt, paidAt
OrderItem	id, orderId, ticketBatchId, quantity, unitPrice
Ticket	id, orderItemId, code (UUID), status, checkedInAt

6.2 Estados de Order e Ticket

Entidade	Transições de estado
Order	PENDING → PAID PENDING → CANCELLED PENDING → EXPIRED
Ticket	VALID → USED VALID → CANCELLED
Event	DRAFT → PUBLISHED → CANCELLED PUBLISHED → FINISHED

7. Arquitetura e organização do código

O projeto adota arquitetura em camadas tradicional, sem DDD ou hexagonal. O objetivo é manter o código direto, navegável e alinhado ao padrão usado em grande parte dos projetos Spring Boot em produção.

```
src/main/java/com/seunome/eventhub/  
■■■■ controller/      → endpoints REST  
■■■■ service/         → regras de negócio  
■■■■ repository/      → interfaces Spring Data JPA  
■■■■ entity/          → entidades JPA  
■■■■ dto/             → request e response objects  
■■■■ mapper/          → conversão entity ↔ dto  
■■■■ config/          → SecurityConfig, RedisConfig, KafkaConfig...  
■■■■ exception/       → exceptions e ControllerAdvice  
■■■■ messaging/       → produtores e consumidores Kafka  
■■■■ scheduling/      → jobs agendados (@Scheduled)  
■■■■ EventHubApplication.java
```

Princípios de design

- Controllers finos: apenas recebem, validam e delegam para o service.
- Services concentram a regra de negócio e orquestram repositórios e mensageria.
- DTOs separam o contrato da API das entidades de persistência.
- Exceções de negócio são tratadas em um @ControllerAdvice global.
- Nenhuma lógica de negócio em controllers ou repositórios.

8. Stack tecnológico

Categoria	Tecnologias
Linguagem	Java 21
Framework	Spring Boot 3.x (Web, Data JPA, Security, Kafka, Validation)
Banco de dados	PostgreSQL
Cache / locks	Redis (Spring Data Redis ou Redisson)
Mensageria	Apache Kafka
Migrations	Flyway
Mapeamento	MapStruct
Documentação	SpringDoc OpenAPI (Swagger UI)
Testes	JUnit 5, Mockito, AssertJ, Testcontainers
Build	Maven ou Gradle
Containerização	Docker, Docker Compose
CI/CD (fase final)	GitHub Actions

9. Endpoints principais (API)

A lista abaixo cobre os principais endpoints. Endpoints completos ficam documentados via OpenAPI ao subir a aplicação.

9.1 Autenticação

Método	Path	Descrição
POST	/auth/register	Cadastra novo usuário
POST	/auth/login	Autentica e retorna tokens
POST	/auth/refresh	Renova o access token

9.2 Eventos

Método	Path	Descrição
GET	/events	Lista pública (com filtros e paginação)
GET	/events/{id}	Detalhe de um evento
POST	/events	Cria evento (ORGANIZER)
PUT	/events/{id}	Atualiza evento (ORGANIZER, dono)
POST	/events/{id}/publish	Publica evento
POST	/events/{id}/cancel	Cancela evento

9.3 Lotes de ingressos

Método	Path	Descrição
GET	/events/{id}/batches	Lista lotes do evento
POST	/events/{id}/batches	Cria lote (ORGANIZER)
PUT	/batches/{id}	Atualiza lote

9.4 Pedidos e ingressos

Método	Path	Descrição
POST	/orders	Cria pedido (CUSTOMER)
GET	/orders/{id}	Consulta pedido
GET	/orders/me	Lista pedidos do usuário autenticado
POST	/orders/{id}/pay	Dispara processamento de pagamento (simulado)
GET	/tickets/me	Lista ingressos do usuário
POST	/tickets/{code}/check-in	Faz check-in pelo código

10. Roadmap de implementação por fases

O projeto deve ser construído de forma incremental. Cada fase termina com a aplicação rodando, testada e versionada no Git. Não pule para a próxima fase sem completar a anterior.

Fase 1 — MVP funcional

- Setup do projeto Spring Boot e Docker Compose com PostgreSQL.
- Migrations com Flyway desde o primeiro commit.
- Cadastro e autenticação de usuários com JWT.
- CRUD de eventos com regras de papel e status.
- CRUD de lotes de ingressos.
- Listagem pública com paginação, ordenação e filtros.
- Bean Validation em todos os DTOs.
- @ControllerAdvice global com Problem Details.
- Documentação OpenAPI ativa.

Fase 2 — Banco de dados sério

- Análise de queries com EXPLAIN ANALYZE.
- Criação de índices compostos e parciais.
- Uso de @EntityGraph para evitar N+1.
- Soft delete em eventos.
- Auditoria com @CreatedDate e @LastModifiedDate.
- Cache de listagem pública com Redis (@Cacheable e @CacheEvict).
- Rate limiting de login com Redis.

Fase 3 — Testes automatizados

- Testes unitários dos services com Mockito.
- Testes de integração com Testcontainers (Postgres real).
- Testes de regras críticas: estoque, transições de status, permissões.
- Teste de concorrência: 100 threads tentando comprar o último ingresso.

Fase 4 — Compra, mensageria e concorrência

- Endpoint POST /orders com criação atômica e expiração.
- Implementação A: lock pessimista no banco (SELECT FOR UPDATE).
- Implementação B: lock distribuído com Redisson.
- Comparação documentada entre as duas estratégias.
- Produção de evento OrderCreated no Kafka.

- Consumer simula pagamento e publica PaymentProcessed.
- Consumer atualiza pedido e emite tickets.
- Padrão Outbox para garantir consistência banco/Kafka.
- Idempotência nos consumers com chave de mensagem.
- Configuração de retry e Dead Letter Queue.
- Job @Scheduled para expirar pedidos PENDING e devolver estoque.
- Endpoint de check-in com tratamento de concorrência.

Fase 5 (opcional) — Deploy e CI/CD

- Dockerfile multi-stage para imagem enxuta.
- Pipeline GitHub Actions: build, testes, análise estática, push de imagem.
- Deploy na AWS (ECS, EC2 + RDS ou Elastic Beanstalk).
- Logs em CloudWatch.

11. Critérios de aceitação

Um projeto considerado **concluído** deve atender os seguintes critérios antes de ser usado como portfólio:

- Aplicação sobe localmente com no máximo dois comandos.
- Todas as rotas autenticadas exigem JWT válido.
- Não há regras de negócio em controllers.
- Não há credenciais ou segredos em arquivos versionados.
- Existe pelo menos um teste de integração para cada fluxo crítico.
- O teste de concorrência prova que estoque nunca fica negativo.
- Pedidos expirados retornam quantidade ao lote automaticamente.
- Ingressos não podem sofrer duplo check-in.
- Documentação OpenAPI completa e acessível.
- README explica decisões técnicas, não apenas como rodar.

12. Entregáveis e documentação

O projeto deve ser entregue em um repositório público com:

- Código-fonte versionado com commits pequenos e descritivos.
- README com: descrição, stack, como rodar, decisões técnicas, trade-offs.
- Diagrama de arquitetura (Excalidraw ou Mermaid).
- Diagrama do modelo de dados (ER).

- Coleção de requisições exportada (Postman, Insomnia ou Bruno).
- Arquivo docker-compose.yml funcional.
- Scripts de migração Flyway organizados.
- Guia rápido para popular dados de exemplo (data.sql ou endpoint /seed).

Dica: recrutadores tendem a abrir o README antes do código. Um README bem escrito, com seção de decisões técnicas e trade-offs, vale mais que cinco features extras escondidas no código.

13. Próximos passos sugeridos

Após concluir o EventHub, são caminhos naturais para continuar evoluindo:

- Quebrar a aplicação em dois serviços (eventos e pagamentos) e estudar microsserviços.
- Introduzir observabilidade: Prometheus, Grafana, tracing distribuído (OpenTelemetry).
- Estudar arquitetura hexagonal refatorando um módulo para entender o problema que ela resolve.
- Adicionar uma camada de notificações por e-mail consumindo eventos do Kafka.
- Implementar relatórios e dashboards para o organizador (queries agregadas).
- Praticar system design clássico: encurtador de URL, feed, sistema de notificações.

— Fim do documento —